You said:

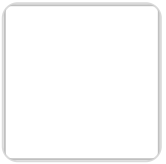
Lab_4_VibeCoding_Student_handout....
PDF

https://github.com/SETAPESU26/45_snake help me finish this assignment

Worked for 7s

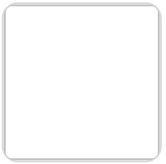
ChatGPT said:

You said:



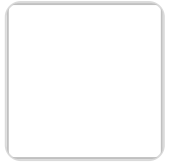
ChatGPT said:

You said:



ChatGPT said:

You said:



ChatGPT said:

You said:

Snake Game

This project is a terminal-based Snake game using ****Pygame****. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

What's Provided

A partially working version of a snake game with:

- Grid-based snake movement controlled by the player
- Food spawning and snake growth
- Score display

You are expected to ****analyze****, ****interact with an AI assistant****, and ****complete/fix**** the game to make it fully functional.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Clone the repo or download the project folder.
2. Make sure you have Python 3.10+ installed.
3. Install dependencies:

```bash
pip install -r requirements.txt
```

4. Run the game:

```bash
python main.py
```

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

> The snake can die instantly and unfairly if you tap the opposite direction key while moving (e.g. pressing Left while already moving Right), because the new head moves straight into the segment behind it. Investigate and enhance collision accuracy so direction changes and self/wall collisions behave fairly.

Task 2: Implement Game Over Condition

> Add a screen that displays the final score once the snake hits a wall or itself, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

> After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard speed), or exit.

Task 4: Add Sound Feedback

> Add basic sound effects for eating food and for the game-over moment.

Expected Behavior

- Smooth, grid-based snake movement using arrow keys or `WASD`
- Snake grows by one segment each time it eats food
- Food never spawns on top of the snake
- Score updates as food is eaten
- Game ends (and optionally restarts) when the snake hits a wall or itself

---

Folder Structure

```
snake-main/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── snake.py
│   └── food.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay ****before**** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay ****after**** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

ChatGPT said:

You said:

```
import pygame

class Snake:
    def __init__(self, x, y, cell_size):
        self.cell_size = cell_size
        # body is a list of (x, y) grid-cell positions, head is body[0]
        self.body = [(x, y), (x - 1, y), (x - 2, y)]
        self.direction = (1, 0)  # moving right
        self.grow_pending = False

    def set_direction(self, dx, dy):
        # NOTE: does not currently guard against reversing directly
        # into the segment behind the head.
        self.direction = (dx, dy)
```

```
def move(self):
    head_x, head_y = self.body[0]
    dx, dy = self.direction
    new_head = (head_x + dx, head_y + dy)

    self.body.insert(0, new_head)
    if self.grow_pending:
        self.grow_pending = False
    else:
        self.body.pop()

def grow(self):
    self.grow_pending = True

def head_rect(self):
    x, y = self.body[0]
    return pygame.Rect(x * self.cell_size, y * self.cell_size, self.cell_size, self.cell_size)

def segment_rects(self):
    return [
        pygame.Rect(x * self.cell_size, y * self.cell_size, self.cell_size, self.cell_size)
        for (x, y) in self.body
    ]

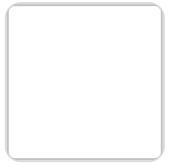
def collides_with_self(self):
    head = self.body[0]
    return head in self.body[1:]

def collides_with_wall(self, grid_width, grid_height):
```

```
x, y = self.body[0]
return x < 0 or y < 0 or x >= grid_width or y >= grid_height
```

ChatGPT said:

You said:



ChatGPT said:

You said:

```
import pygame
from .snake import Snake
from .food import Food

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
```

```
        self.cell_size = 20
        self.grid_width = width // self.cell_size
        self.grid_height = height // self.cell_size

        self.snake = Snake(self.grid_width // 2, self.grid_height // 2, self.cell_size)
        self.food = Food(self.grid_width, self.grid_height, self.cell_size)

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)

        self.moves_per_second = 8
        self._frame_counter = 0

        self.game_over = False
        self._game_over_logged = False

    def handle_keydown(self, key):
        # Direction changes are applied immediately on key press.
        if key in (pygame.K_UP, pygame.K_w):
            self.snake.set_direction(0, -1)
        elif key in (pygame.K_DOWN, pygame.K_s):
            self.snake.set_direction(0, 1)
        elif key in (pygame.K_LEFT, pygame.K_a):
            self.snake.set_direction(-1, 0)
        elif key in (pygame.K_RIGHT, pygame.K_d):
            self.snake.set_direction(1, 0)

    def handle_input(self):
        # Reserved for continuously-held-key input (not used for a
        # grid-based snake, but kept here to mirror the engine's shape).
```

```
        pass

    def update(self):
        if self.game_over:
            return

        self._frame_counter += 1
        frames_per_move = max(1, 60 // self.moves_per_second)
        if self._frame_counter < frames_per_move:
            return
        self._frame_counter = 0

        self.snake.move()

        if self.snake.collides_with_wall(self.grid_width, self.grid_height):
            self.game_over = True
            return

        if self.snake.collides_with_self():
            self.game_over = True
            return

        if self.snake.head_rect().colliderect(self.food.rect()):
            self.snake.grow()
            self.score += 1
            self.food.respawn(self.snake.body)

    def render(self, screen):
        # Draw food
        pygame.draw.rect(screen, RED, self.food.rect())
```

```
# Draw snake
for rect in self.snake.segment_rects():
    pygame.draw.rect(screen, GREEN, rect)

# Draw score
score_text = self.font.render(f"Score: {self.score}", True, WHITE)
screen.blit(score_text, (10, 10))

if self.game_over and not self._game_over_logged:
    # NOTE: no proper game-over screen yet - see Task 2 in the README.
    print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

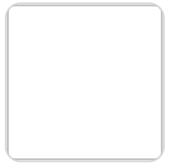
ChatGPT said:

You said:

just give me the entire code to replace

ChatGPT said:

You said:



the game ran

ChatGPT said:

You said:

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 600
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake - Pygame Version")

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
```

```
        SCREEN.fill(BLACK)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            if event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

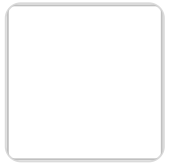
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

ChatGPT said:

You said:



ChatGPT said:

You said:

it runs

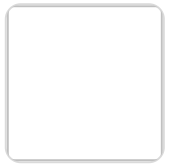
ChatGPT said:

You said:

tested

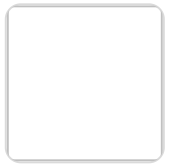
ChatGPT said:

You said:



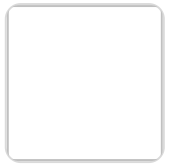
ChatGPT said:

You said:



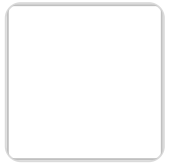
ChatGPT said:

You said:



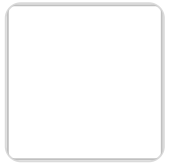
ChatGPT said:

You said:



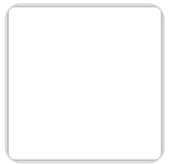
ChatGPT said:

You said:



ChatGPT said:

You said:



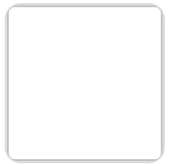
ChatGPT said:

You said:

how do I return to terminal

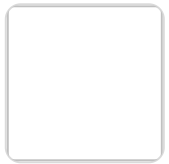
ChatGPT said:

You said:



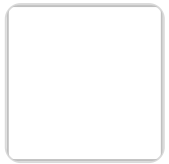
ChatGPT said:

You said:



ChatGPT said:

You said:



ChatGPT said:

You said:

i am not able to record

ChatGPT said:

You said:

Xbox Game Bar records my desktop but not the Snake game.

ChatGPT said:

You said:

i dont wanna donlowad all that

ChatGPT said:

You said:

i recorded on my phone and ssent to my laptop now lets finish the deliverables

ChatGPT said:

You said:

done

ChatGPT said:

You said:

done both videos

ChatGPT said: